

Pen testing course

Rubric 3

Vulnerability Assessment

Engagement title: Home Lab Internal Network Assessment **Phase:** PTES Phase 4 (Vulnerability Analysis) **Document version:** 1.0 **Classification:** Confidential — Lab Use Only
Prepared by: [Your name], DAE Cybersecurity Program **Date:** [Fill in] **Related documents:** Engagement Plan (Rubric 1), Reconnaissance Report (Rubric 2)

This is the vulnerability-analysis stage of the same engagement. Scope and authorization carry forward from Rubric 1; the discovered services carry forward from Rubric 2. Placeholders in [brackets] are yours to fill.

Table of Contents

1. Executive Summary
 2. Scope & Methodology
 3. Tools & Configuration
 4. Automated Vulnerability Scanning
 5. Manual Vulnerability Verification
 6. Detailed Findings
 7. Risk Analysis & Prioritization
 8. Remediation Recommendations
 9. Evidence Log
 10. Appendix A — CVSS Primer
 11. Appendix B — Command Reference & References
-

1. Executive Summary

(Write this section for a non-technical reader — a manager or client. No jargon, no commands. Summarize risk and business impact.)

An authorized vulnerability assessment was performed against the in-scope host **192.168.78.5** in the lab environment. The assessment combined automated scanning with manual verification to identify and confirm security weaknesses.

The assessment identified **[N] confirmed vulnerabilities**. The most significant is the use of the **Telnet protocol for remote administration**, which transmits usernames and passwords with no encryption. Any party with access to the local network could intercept administrator credentials and gain full control of the system. This finding is rated **High** risk and should be remediated immediately by replacing Telnet with SSH.

Findings by severity:

Severity	Count
Critical	[0]
High	[1]
Medium	[]
Low	[]

Top recommendation: Disable Telnet and adopt SSH for all remote access.

2. Scope & Methodology

In scope: **192.168.78.5** only (per the signed authorization in Rubric 1).

Methodology — a two-stage process, because scanners alone are not trustworthy:

1. **Automated scanning** casts a wide net quickly across all services, matching them against a database of known vulnerabilities.
2. **Manual verification** confirms each candidate finding by hand. Scanners produce *false positives* (flagging issues that aren't real) and *false negatives* (missing real ones). A finding is only reported after a human confirms it, which is what separates a professional assessment from a raw scan dump.

Each confirmed finding is then scored with **CVSS 3.1** and prioritized by risk.

3. Tools & Configuration

Choose one automated scanner as your primary tool. Any of these satisfies "appropriate tools"; Nessus Essentials produces the most portfolio-friendly report.

Tool	Type	Where it runs	Notes
Nmap NSE (<code>--script vuln</code>)	Lightweight scanner	Windows or Linux tester	Uses the Nmap you already have; fast to start
Nessus Essentials	Full vulnerability scanner	Windows or Ubuntu	Free for up to 16 IPs; professional HTML/PDF reports
OpenVAS / Greenbone (GVM)	Full vulnerability scanner	Kali / Docker	Fully open-source; heavier setup

Proper scope configuration (critical for the rubric): in whichever tool you use, set the scan target to **192.168.78.5 only** — never the whole subnet or anything out of scope. Document the exact target, scan policy, and date/time you ran it. Scanning outside the authorized scope, even by leaving a `/24` in the target box, is an ethics and authorization failure.

4. Automated Vulnerability Scanning

4.1 Option A — Nmap NSE (quick start)

```
sudo nmap -sV --script vuln 192.168.78.5 -oN vuln-scan.txt
```

- `-sV` = detect service versions (the vuln scripts need this).
- `--script vuln` = run all scripts in Nmap's "vuln" category, which check discovered services against known issues.
- `-oN vuln-scan.txt` = save the output as evidence.

The output lists candidate vulnerabilities per port. Treat every result as *unconfirmed* until Section 5.

4.2 Option B — Nessus Essentials (recommended for the report)

1. Register for a free Nessus Essentials activation code at tenable.com and install it (Windows or Ubuntu).
2. Open the web console at <https://localhost:8834>.
3. Create a new scan → **Basic Network Scan**.
4. Set **Targets** to [192.168.78.5](#) (single host — proper scope).
5. Launch the scan, let it finish, and open the results.
6. Nessus groups findings by severity (Critical/High/Medium/Low/Info) with a CVSS score and description for each.
7. **Export** the report (HTML or PDF) — this becomes a strong evidence artifact.

4.3 What to record

For each automated finding: the plugin/script name, the affected port, the severity the tool assigned, and the CVE reference if given. Put these in a raw findings table before verification.

5. Manual Vulnerability Verification

This is the section that demonstrates real skill. For each automated finding, confirm it independently. Below is the methodology applied to the flagship finding.

5.1 Verify the Telnet cleartext weakness (Finding VA-01)

Step 1 — Confirm the service and banner (banner grabbing):

```
telnet 192.168.78.5 23
```

The connection opening and presenting a login prompt confirms Telnet is live on port 23.

Step 2 — Prove credentials travel in cleartext (packet capture):

On the Ubuntu VM, capture Telnet traffic while logging in from the laptop:

```
sudo tcpdump -i any -A 'tcp port 23' -w telnet-verify.pcap
```

- **-A** prints packet contents as ASCII so text is readable live.
- **-w [telnet-verify.pcap](#)** also saves the capture for Wireshark and for evidence.

Then log in from the laptop with a test username and password. In the tcpdump output (or by opening the [.pcap](#) in Wireshark → right-click a packet → **Follow** → **TCP Stream**), the

username and password appear as **readable plaintext**. That capture is your verification — it proves the vulnerability is real, not a scanner guess.

5.2 General verification techniques for other findings

Finding type	How to verify manually
Outdated service version	Take the exact version from <code>-sV</code> , look it up in the National Vulnerability Database (nvd.nist.gov) and Exploit-DB to confirm a real CVE applies
Weak/default credentials	Attempt a login with known defaults (in scope only)
Missing encryption	Capture traffic and confirm data is unencrypted (as in 5.1)
False positive	Reproduce the tool's claim by hand; if you can't, mark it not confirmed and exclude it

5.3 Verification results

Finding	Automated result	Manually confirmed?	Evidence
VA-01 Telnet cleartext	Flagged	Yes	<code>telnet-verify.py</code> , TCP stream screenshot
[VA-02 ...]	[]	[]	[]

6. Detailed Findings

Finding VA-01 — Cleartext Credential Transmission via Telnet

Field	Value
Severity	High
CVSS 3.1 base score	7.5

Field	Value
CVSS vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Affected asset	192.168.78.5, port 23/tcp
Status	Confirmed (manual verification, Section 5.1)

Description: The host runs a Telnet service that transmits all session data — including authentication credentials — without encryption. An attacker positioned on the local network can intercept and read valid credentials.

Evidence: Packet capture `telnet-verify.pcap` showing the username and password in plaintext (see Evidence Log).

Impact: The intercepted account (`vboxuser`) holds `sudo` privileges, so credential disclosure leads directly to full administrative compromise of the host.

Vector justification: No privileges or special conditions are required to intercept traffic on the segment (`PR:N`, `AC:L`), and confidentiality impact is complete because credentials are fully exposed (`C:H`). *Note on judgment:* some assessors score interception as Adjacent (`AV:A`) with user interaction required (`UI:R`), which lowers the base score to Medium. Given the exposed account's admin rights and the flat lab network, **High** is the appropriate contextual rating — see Section 7.

Finding VA-02 — *[e.g., Outdated service version / open unnecessary port]*
(Fill in from your scan results using the same structure.)

7. Risk Analysis & Prioritization

Risk is not just the raw CVSS score — it's **likelihood × impact** in *this* environment. Use the matrix to justify each rating.

	Low Impact	Medium Impact	High Impact
High Likelihood	Medium	High	Critical
Medium Likelihood	Low	Medium	High

	Low Impact	Medium Impact	High Impact
Low Likelihood	Low	Low	Medium

Prioritized findings:

Priority	Finding	CVSS	Likelihood	Impact	Final risk	Justification
1	VA-01 Telnet cleartext	7.5	High	High	High	Easy to exploit by anyone on the segment; exposes an admin account
2	[VA-02]	[]	[]	[]	[]	[]

Justification narrative: VA-01 is prioritized first because both factors are high — interception requires only local network access and no special skill (high likelihood), and the exposed credentials grant administrative control (high impact). This combination is why it outranks findings that may have a similar raw score but lower real-world likelihood.

8. Remediation Recommendations

Finding	Recommendation	Verification after fix
VA-01	Disable Telnet and use SSH instead: install SSH (<code>sudo apt install openssh-server</code>), confirm login works over port 22, then disable Telnet (<code>sudo update-inetd --disable telnet</code>) and remove it (<code>sudo apt remove inetutils-telnetd</code>).	Re-scan: port 23 should show closed ; capture SSH traffic and confirm it is encrypted (unreadable)

Finding	Recommendation	Verification after fix
[VA-02]	[patch/upgrade/close port/harden config]	[re-scan to confirm resolved]

General hardening recommendations: keep services patched, close unnecessary ports, enforce strong credentials, and segment the network so a single host compromise doesn't expose everything.

9. Evidence Log

- ☐ `vuln-scan.txt` — automated Nmap NSE output
 - ☐ Nessus/OpenVAS exported report (HTML/PDF)
 - ☐ `telnet-verify.pcap` — packet capture proving cleartext credentials
 - ☐ Screenshot of the Wireshark **Follow TCP Stream** showing plaintext login
 - ☐ Screenshot of scanner results grouped by severity
 - ☐ NVD/Exploit-DB lookup screenshots for any version-based findings
-

Appendix A — CVSS Primer

CVSS (Common Vulnerability Scoring System) rates severity 0–10 from a **vector string** of metrics:

Metric	Meaning	Common values
AV — Attack Vector	Where the attacker must be	Network / Adjacent / Local / Physical
AC — Attack Complexity	How hard to exploit	Low / High
PR — Privileges Required	Access needed first	None / Low / High
UI — User Interaction	Does a victim have to act?	None / Required
C / I / A — Impact	Confidentiality, Integrity, Availability	None / Low / High

Severity bands: 0.1–3.9 Low · 4.0–6.9 Medium · 7.0–8.9 High · 9.0–10.0 Critical. Use the FIRST.org calculator (first.org/cvss/calculator/3.1) to generate and justify each score.

Appendix B — Command Reference & References

Goal	Command
Automated NSE vuln scan	<code>sudo nmap -sV --script vuln 192.168.78.5 -oN vuln-scan.txt</code>
Banner grab	<code>telnet 192.168.78.5 23</code>
Verify cleartext	<code>sudo tcpdump -i any -A 'tcp port 23' -w telnet-verify.pcap</code>
Confirm fix	<code>nmap -p 23,22 192.168.78.5</code>

- NIST SP 800-115, *Technical Guide to Information Security Testing and Assessment*
- CVSS v3.1 Specification & Calculator — first.org/cvss
- National Vulnerability Database — nvd.nist.gov
- Exploit Database — exploit-db.com
- Tenable Nessus documentation — docs.tenable.com

This report is for authorized security testing in a controlled lab environment only.

Work documentation

Click **Advanced...**, then click through to proceed (the button usually reads something like "Accept the Risk and Continue"). That'll take you to the actual Nessus setup screen.

This is a one-time thing per browser profile — once you click through, Firefox remembers the exception and won't ask again for this site.

If clicking Advanced doesn't reveal a way through (some strict configurations hide it), the fallback is confirming the service is actually listening before fighting with the browser further. In a Kali terminal:

```
sudo ss -tlnp | grep 8834
```

You want to see it bound to `127.0.0.1:8834` or `0.0.0.0:8834`. If nothing shows up, the service isn't actually up yet even though `systemctl start` didn't error — worth checking `sudo systemctl status nessusd.service` again to confirm it says `active (running)`.

Click on advanced

Someone could be trying to impersonate the site and you should not continue.

Websites prove their identity via certificates. Firefox does not trust localhost:8834 because its certificate issuer is unknown, the certificate is self-signed, or the server is not sending the correct intermediate certificates.

Error code: [SEC_ERROR_UNKNOWN_ISSUER](#)

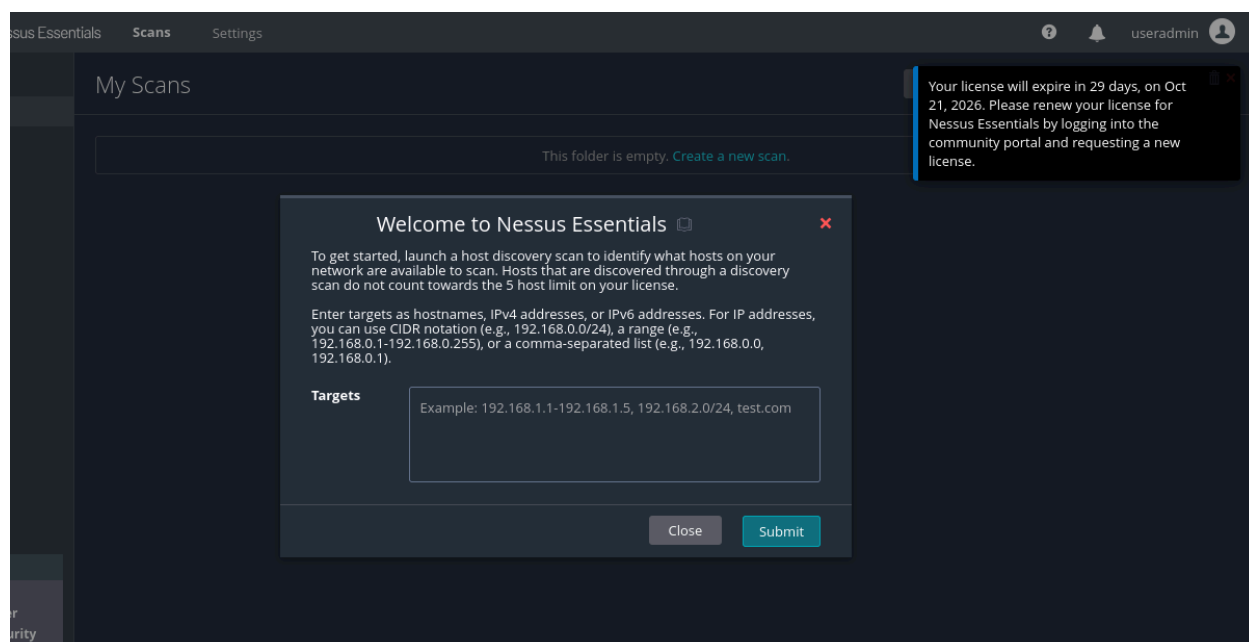
[View Certificate](#)

Go Back (Recommended)

Accept the Risk and Continue

Then nessus will take you here

```
—(kyloren@kyloren)-[~/Downloads]
—$ ls -l | grep Ness
-rw-rw-r-- 1 kyloren kyloren 74304388 Aug 27 07:47 Nessus-10.12.4-debian10_amd64.deb
```



My Host Discovery Scan Results

Nessus found the following hosts listed below from your list of targets (192.168.78.5).

To launch your first basic network scan, select the hosts you want to scan. These hosts count towards the 5 host limit on your license.

☐	IP	DNS
☐	192.168.78.5	

Discovering Hosts...

Back

Run Scan

My Host Discovery Scan Results

Nessus found the following hosts listed below from your list of targets (192.168.78.5).

To launch your first basic network scan, select the hosts you want to scan. These hosts count towards the 5 host limit on your license.

☒	IP	DNS
☒	192.168.78.5	

Discovery Complete!

Back

Run Scan

← → ↺ 🏠

localhost:8834/#/scans/reports/8/hosts

☆

🔔

☰

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

tenableNessus EssentialsScansSettings

🔔🔔🔔useradmin

FOLDERS

My Scans1

All Scans

Trash

RESOURCES

Policies

Plugin Rules

Tenable News

WordPress Loops & Logic - Reflected XSS

My Basic Network Scan

← Back to My Scans

Hosts1Vulnerabilities1History1

Filter Search Hosts1 Host

Host	Auth	Vulnerabilities	%
192.168.78.5	N/A	1	2%

Scan Details

Policy: Basic Network Scan

Status: Running

Severity Base: CVSS v3.0

Scanner: Local Scanner

Start: Today at 11:50 PM

Vulnerabilities

Critical

High

Medium

Low

Info

Your license will expire in 29 days, on Oct 21, 2026. Please renew your license for Nessus Essentials by logging into the community portal and requesting a new license.

Upgrade to Nessus Essentials Plus for real-time plugin updates, more IPs, and reporting capabilities!UPGRADE NOW

My Basic Network Scan

ConfigureAudit

← Back to My Scans

Hosts1Vulnerabilities14History1

Filter Search Vulnerabilities14 Vulnerabilities

Sev	CVSS	VPR	EPSS	Name	Family	Count		
LOW	2.1 *	3.2	0.3159	ICMP Timestamp Request Remote Date Disclosure	General	1	🔍	✎
INFO				Common Platform Enumeration (CPE)	General	1	🔍	✎
INFO				Device Type	General	1	🔍	✎
INFO				Ethernet Card Manufacturer Detection	Misc.	1	🔍	✎
INFO				Ethernet MAC Addresses	General	1	🔍	✎
INFO				FTP Server Detection	Service detection	1	🔍	✎
INFO				mDNS Detection (Local Network)	Service detection	1	🔍	✎
INFO				Nessus Scan Information	Settings	1	🔍	✎
INFO				Nessus SYN scanner	Port scanners	1	🔍	✎
INFO				OS Fingerprints Detected	General	1	🔍	✎
INFO				OS Identification	General	1	🔍	✎
INFO				TCP/IP Timestamps Supported	General	1	🔍	✎
INFO				Traceroute Information	General	1	🔍	✎

Hosts 1

Vulnerabilities 14

History 1

ICMP Timestamp Request Remote Date Disclosure

LOW ICMP Timestamp Request Remote Date Disclosure

Description

The remote host answers to an ICMP timestamp request. This allows an attacker to know the date that is set on the targeted machine, which may assist an unauthenticated, remote attacker in defeating time-based authentication protocols.

Timestamps returned from machines running Windows Vista / 7 / 2008 / 2008 R2 are deliberately incorrect, but usually within 1000 seconds of the actual system time.

Solution

Filter out the ICMP timestamp requests (13), and the outgoing ICMP timestamp replies (14).

Output

The remote clock is synchronized with the local clock.

To see debug logs, please visit individual host

Port Hosts

0 / icmp 192.168.78.5

Plugin Details

Severity: Low
ID: 10114
Version: 1.56
Type: remote
Family: General
Published: August 1, 1999
Modified: October 7, 2024

VPR Key Drivers

Threat Recency: No recorded events
Threat Intensity: Very Low
Exploit Code Maturity: PoC
Age of Vuln: 730 days +
Product Coverage: Very High
CVSSv3 Impact Score: 1.4
Threat Sources: No recorded events

Risk Information

Vulnerability Priority Rating (VPR): 3.2
Exploit Prediction Scoring System (EPSS): 0.3159
Risk Factor: Low
CVSS v2.0 Base Score: 2.1
CVSS v2.0 Vector: CVSS2#AV:L/AC:L/Au:N/C:P/IN/A/N

Vulnerability Information

Plugin Details



Severity: Low
ID: 10114
Version: 1.56
Type: remote
Family: General
Published: August 1, 1999
Modified: October 7, 2024

VPR Key Drivers

Threat Recency: No recorded events
Threat Intensity: Very Low
Exploit Code Maturity: PoC
Age of Vuln: 730 days +
Product Coverage: Very High
CVSSV3 Impact Score: 1.4
Threat Sources: No recorded events

Risk Information

Vulnerability Priority Rating (VPR): 3.2
Exploit Prediction Scoring System (EPSS): 0.3159
Risk Factor: Low
CVSS v2.0 Base Score: 2.1
CVSS v2.0 Vector: CVSS2#AV:L/AC:L/Au:N/C:P/I:N/A:N

Vulnerability Information

Good news — this is one of the easier ones on your list, and Nessus's own **Solution** field already tells you exactly what to do. It just needs to actually be applied on Ubuntu.

What it means: ICMP timestamp requests (type 13) get an ICMP timestamp reply (type 14) telling anyone who asks what time the system clock reads. It's rated "Low" because on its own it doesn't compromise anything — it's pure information disclosure that could theoretically help someone target time-based authentication schemes in rare setups, but it's not a path to actual access by itself.

The fix — block ICMP timestamp on Ubuntu:

```
sudo iptables -A INPUT -p icmp --icmp-type timestamp-request -j DROP
sudo iptables -A OUTPUT -p icmp --icmp-type timestamp-reply -j DROP
```

This takes effect immediately but won't survive a reboot unless saved:

```
sudo apt install iptables-persistent -y
sudo netfilter-persistent save
```

If you're using ufw instead, it doesn't expose ICMP-type filtering through the normal `ufw allow/deny` syntax — you'd have to edit `/etc/ufw/before.rules` and insert a DROP rule for `icmp-type 13` ahead of the existing ICMP accept rules, then `sudo ufw reload`. The iptables commands above are simpler and work regardless of ufw's state.

Verify it took:

```
sudo iptables -L -n | grep icmp
```

Then re-run the Nessus scan against `192.168.78.5`. This finding should disappear, or its Output field should stop showing "The remote clock is synchronized with the local clock" since the target will no longer respond at all.

Worth keeping in perspective: this is about as low-stakes as findings get — informational disclosure, no compromise path on its own. Fine to practice remediating for the lab, but if you're meant to prioritize across all 12, this one can sit near the bottom. If you want, share the rest of the findings list and I can help you sort which actually deserve attention first.